How Secure Is a Truncated SHA-2 Digest?

Course report

Truncated SHA-256, birthday collisions,
and a signature that checks only a prefix

Student	Your Name
ID	Student ID
Course	Data Security
Date	November 8, 2026

Problem statement: `assignment/problem/problem.tex`

Contents

1 Introduction **1**

2 Background **1**

2.1 SHA-2 1

2.2 Truncation and significant bits 1

2.3 Birthday bound 1

2.4 The signature on a prefix 1

3 Alice's investigation **1**

4 Collision search **2**

4.1 Hash table 2

4.2 Floyd's cycle-finding algorithm 2

4.3 Time–memory tradeoff 2

4.4 Parallel search 2

5 Experiments **2**

5.1 Setup 2

5.2 Results 2

5.3 What limited the next prefix length 3

6 Meaningful messages **3**

7 Security conclusion **3**

8 Conclusion **3**

A Colliding inputs **3**

B Diagrams of the three searches **4**

References **4**

1 Introduction

State the question in your own words. A signature that verifies $H_n(x) = \text{prefix}_n(H(x))$ accepts every message with that same n-bit prefix. Say which n you studied and what you concluded about the signature.

2 Background

2.1 SHA-2

Identify the member of the family you hashed (SHA-256 unless you changed it), its output length, and the properties the full function is designed to provide: collision resistance and preimage resistance. Name where a digest of this kind is used (signatures, TLS, blockchains) only as far as it matters for the attack.

2.2 Truncation and significant bits

Define $H_n(x) = \text{prefix}_n(H(x))$ and state that the prefix is the most significant n bits, the leading bits when the digest is written in hex. A short example, even one byte, is enough to fix MSB and LSB.

2.3 Birthday bound

Give the collision condition $x \neq x'$ and $H_n(x) = H_n(x')$, and the generic cost $O(2^{n/2})$. Record the lecture's reading of that cost: truncation leaves at most $n/2$ bits of collision security, short prefixes such as $n = 32$ and $n = 64$ are practical, and $n \geq 128$ is the example of a length that is still large.

2.4 The signature on a prefix

Say what the verifier computes and what it compares. Then say what follows when two messages share a prefix: a signature on one verifies on the other.

3 Alice's investigation

Summarize the poisoned-message setting you are replaying. Two documents that a reader accepts as different messages share one digest, so the signature transfers. Cite Daum and Lucks, and point at the published MD5 pair only as the historical case. Keep this section to the analogy. The measurements belong in Section 5.

4 Collision search

Open with the search problem: sample inputs until two of them share an n-bit prefix. Then one subsection per approach you actually ran. Delete the subsections you did not use, and add one for each further idea (distinguished points, a different cycle finder, a GPU search, and so on).

4.1 Hash table

Describe the key you store, the value you store beside it, and the test that declares a collision. Give the memory per entry and the n at which the table stopped fitting. This is the memory-heavy end of the tradeoff.

4.2 Floyd's cycle-finding algorithm

Describe the function you iterate, the two pointers, and the extra work you do after a meeting point to recover a colliding pair. State the memory, which should stay essentially constant, and the number of hashes relative to $2^{n/2}$.

4.3 Time–memory tradeoff

Put the two previous methods on one scale. If you used distinguished points, chains, or another intermediate scheme, specify the parameter that moves memory in one direction and hashing work in the other, and report the settings you tried.

4.4 Parallel search

Describe how the search was split (processes, threads, or machines), how a worker reported a hit, and how the speed compared with one worker. Note any duplicated work.

5 Experiments

5.1 Setup

Machine, core count, RAM, language, and hash implementation. State how you timed a run and how you measured memory. Fix the input format (counter, random block, chosen prefix plus a free block) so a reader can reproduce a pair.

5.2 Results

Fill Table 1. One row per (n, method) you finished. The largest n is the bottom line of the assignment; start at 32 and keep every successful length, including the ones that were easy.

Table 1: Collision searches. Replace the dashes. "Hashes" is the number of SHA-256 evaluations.

n	Method	Hashes	Time	Memory	Prefix
32	hash table	—	—	—	`----`
32	Floyd	—	—	—	`----`
64		—	—	—	`--------`
Largest n reached: _____				method: _____	

5.3 What limited the next prefix length

For the smallest n you did not finish, say whether you ran out of memory, time, or both, and which method came closest. Compare that wall with $2^{n/2}$.

6 Meaningful messages

Leave this section until the lecture that builds a collision into a document a person will actually read. Then describe the two messages, where the free bytes sit, and why a reader sees two different texts while H_n stays the same. Attach the files or quote only enough of them to identify the pair. If that lecture has not happened yet, write one sentence saying so and point ahead to the conclusion you can already draw from Section 5.

7 Security conclusion

Answer the mission directly. For each n you broke, the signature on H_n does not separate the two inputs you found. For the lengths you did not break, give the birthday cost and say whether that cost is still a practical attack or only a theoretical one. Use the lecture's markers: $n = 32$ and $n = 64$ are the short prefixes, and $n \geq 128$ is the example of a prefix that remains large. Close with why the birthday bound, rather than a dedicated SHA-256 break, is the attack that decides this question.

8 Conclusion

Three or four sentences. Largest n, the method that reached it, the memory–time choice that mattered, and the consequence for the signature. State the meaningful-message attack as done, or as waiting on the later lecture.

A Colliding inputs

For every row of Table 1, list the two inputs and the shared prefix. Prefer a path to a file in the repository over a paste of long inputs.

```
n = 32
method:
x  =
x' =
Hn =
```

B Diagrams of the three searches

Each arrow is one SHA-256 evaluation. The next value is the leading bits of that digest, written here in hex. Blue is a value placed in a table. Orange marks the two inputs of a collision. The pictures use a 16-bit prefix so the hex fits on the page. At $n = 32$ the same pictures keep the first eight hex digits.

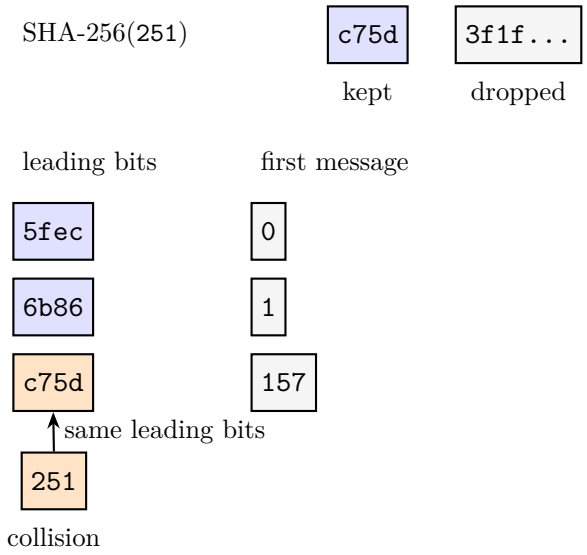


Figure 1: Hash table, with real SHA-256 prefixes of the decimal counters. Every leading prefix is stored with the first message that produced it. `0` and `1` have new prefixes. `157` is stored under `c75d`. `251` produces the same leading digits, so the two messages collide. The digits after `c75d` differ and are not stored.

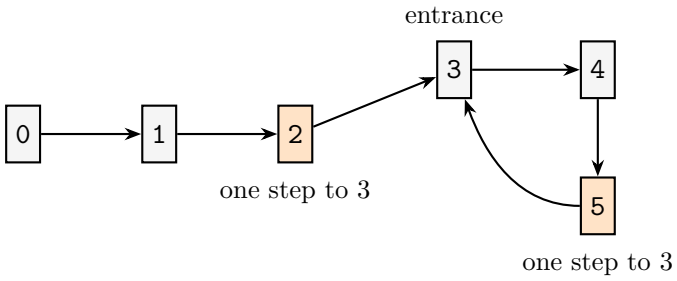


Figure 2: Floyd's algorithm on a short stand-in chain. Each arrow hashes the state and keeps the leading bits; those bits are the next state. The tortoise follows one arrow at a time and the hare follows two, until they meet on the cycle and walk back to the entrance `3`. States `2` and `5` both step onto `3`, so they are the collision. Only the two pointers are kept. No table of states is written.

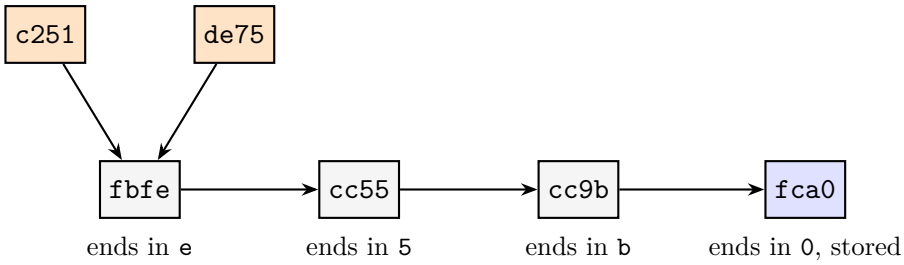


Figure 3: Distinguished points on a real SHA-256 chain, with a 16-bit prefix. `c251` and `de75` both hash to the leading bits `fbfe`. That pair is the collision. The chain continues, and a state is stored only when it ends in the hex digit `0`, here the lowest four bits `0000`. `fbfe`, `cc55`, and `cc9b` fail that test. The chain stops at `fca0`, which is the single table entry. A second chain that ends at the same `fca0` is walked again to recover `c251` and `de75`.

References

[1] Magnus Daum and Stefan Lucks. Hash Collisions (The Poisoned Message Attack). `https://www.sos.cs.ru.nl/applications/courses/security2015/md5collisions/index.html`.

[2] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015.